Baud rate generator:

```
module baudrate_gen(
input pclk,
input presetn,
input [1:0]spi_mode_i,
input spiswai_i,
input [2:0]sppr_i,
input [2:0]spr_i,
input cpol_i,cpha_i,ss_i,

output reg sclk,
output reg mosi_send_sclk_o,
output reg mosi_send_sclk0_o,
output reg miso_recive_sclk_o,
output reg miso_recive_sclk0_o,
output [11:0]BaudrateDivisor_o
);

wire pre_sclk;
reg [11:0]count;
wire spimode;

assign BaudrateDivisor_o =((sppr_i+1'b1)* 2**((spr_i+1'b1)));


assign pre_sclk = (cpol_i==1) ? 1'b1: 1'b0;


//logic for sclk
always @(posedge pclk or negedge presetn)
begin
	if(!presetn)
		sclk<=pre_sclk;
	else if((spi_mode_i==2'b00 || spi_mode_i==2'b01) && !ss_i && !spiswai_i)
	begin
		if(count==((BaudrateDivisor_o)/2)-1'b1)
			sclk<=~sclk;
		else
			sclk<=sclk;
	end
	else
		sclk<=pre_sclk;
end


always  @(posedge pclk or negedge presetn)
begin
	if(!presetn)
		count<=0;
	else if((spi_mode_i==2'b00 || spi_mode_i==2'b01) && !ss_i && !spiswai_i)
	begin
```

```
                if(count==((BaudrateDivisor_o)/2)-1'b1)
                        count<= 0;
                else
                        count<=count+1'b1;
        end
        else
                count<=count;
end


//logic for the gate implementation for cpol and cpha
assign spimode =((~cpha_i & cpol_i ) || (cpha_i & ~cpol_i));



//logic for mosi
always @(posedge pclk or negedge presetn)
begin
        if(!presetn)
        begin
                mosi_send_sclk_o<=0;
                mosi_send_sclk0_o<=0;

        end
        else if (!spimode) begin
                        if(sclk)
                        begin
                                if (count==(BaudrateDivisor_o/2)-2'b1)
                                        mosi_send_sclk_o<=1'b1;
                                else
                                        mosi_send_sclk_o<=1'b0;
                        end
                        else
                                mosi_send_sclk_o<=1'b0;
                end

        else begin
                        if(!sclk) begin
                                if(count==(BaudrateDivisor_o/2)-2'b1)
                                        mosi_send_sclk0_o<=1'b1;
                                else
                                     mosi_send_sclk0_o<=1'b0;
                                end
                         else
                                 mosi_send_sclk0_o<=1'b0;
        end
end

//logic for miso
always @(posedge pclk or negedge presetn)
begin
        if(!presetn)
```

```
        begin
                miso_recive_sclk_o<=0;
                miso_recive_sclk0_o<=0;
        end
        else if (!spimode)
        begin
                if(sclk)
                begin
                        if(count==(BaudrateDivisor_o/2)-1'b1)
                                miso_recive_sclk0_o<=1'b1;
                        else
                                miso_recive_sclk0_o<=1'b0;
                end
                else
                        miso_recive_sclk0_o<=1'b0;
        end
        else begin
                if(!sclk)begin
                        if(count==(BaudrateDivisor_o/2)-1'b1)
                                miso_recive_sclk_o<=1'b1;
                        else
                                miso_recive_sclk_o<=1'b0;
                end
                else
                        miso_recive_sclk_o<=1'b0;
        end
end

endmodule
```

SPI slave select:

```
module
spi_slave_select(pclk,presetn,mstr_i,spiswai_i,spimode_i,send_data_i,baudRateDivisor_i,receive_d
ata_o,ss_o,tip_o);
input pclk,presetn,mstr_i,spiswai_i,send_data_i;
input [1:0]spimode_i;
input [11:0]baudRateDivisor_i;

output reg receive_data_o,ss_o;
output tip_o;

wire [15:0]target;
reg rcv;
reg [15:0]count;

assign target= (4'd8)*(baudRateDivisor_i);
```

```
assign tip_o=~ss_o;

always @(posedge pclk or negedge presetn) begin
	if(!presetn)
		receive_data_o<=1'b0;
	else
		receive_data_o<=rcv;
end

always@(posedge pclk or negedge presetn) begin
	if(!presetn) begin
		ss_o<=1'b1;
		count<=16'hffff;
		rcv<=1'b0;
	end
	else begin
		if(mstr_i && !spiswai_i && (spimode_i==2'b00||spimode_i==2'b01))begin
			if(send_data_i) begin
				ss_o<=1'b0;
				rcv<=1'b0;
				count<=16'd0;

			end


			else if (count<=target-1'b1) begin
				ss_o<=1'b0;
				count<=count+1'd1;
				if(count==target-1'b1)
					rcv<=1'b1;
				else
					rcv<=1'b0;
			end
			else begin
				ss_o<=1'b1;
				rcv<=1'b0;
				count<=16'hffff;
			end
		end
		else begin
			ss_o<=1'b1;
			rcv<=1'b0;
			count<=16'hffff;
		end
	end
end

endmodule
```

Shift Register:

```
module shift_reg(input pclk,presetn,ss_i,send_data_i,
                input lsbfe_i,cpha_i,cpol_i,
                input miso_receive_sclk0,
                input miso_receive_sclk,
                input mosi_send_sclk0,
                input mosi_send_sclk,
                input [7:0]data_mosi_i,
                input miso_i,receive_data_i,

                output reg mosi_o,
                output [7:0]data_miso_o);

        reg [7:0]shift_reg,temp;
        reg [2:0]count,count1,count2,count3;
        wire spimode;

        //spimode for which edge of sclk
        assign spimode=((~cpol_i && cpha_i)||(cpol_i && ~cpha_i));


        //logic for data out in miso
        assign data_miso_o = (receive_data_i)?temp:8'd0;

        //logic for shift_reg

        always @(posedge pclk or negedge presetn) begin
                if(!presetn)
                        shift_reg<=8'd0;
                else begin
                        if(send_data_i)
                                shift_reg<=data_mosi_i;
                        else
                                shift_reg<=shift_reg;
                end
        end

        //logic for count2,count3, temp

        always@(posedge pclk or negedge presetn) begin
                if(!presetn)begin
                        count2<=3'd0;
                        count3<=3'd7;
                        temp<=8'd0;
                end
                else begin
                        if(ss_i) begin
                                count2<=count2;
                                count3<=count3;
```

```
		temp<=temp;
	end
	else begin
		if(spimode) begin
			if(lsbfe_i) begin
				if(count2<=3'd7) begin
					if(miso_receive_sclk0) begin
						count2<=count2+3'd1;
						temp[count2]<=miso_i;
					end
					else begin
						count2<=count2;
						temp[count2]<=temp[count2];
					end
				end
				else begin
					count2<=3'd0;
					temp[count2]<=0;
				end
			end
			else begin
				if(count3>=3'd0) begin
					if(miso_receive_sclk0) begin
						count3<=count3-3'd1;
						temp[count3]<=miso_i;
					end
					else begin
						count3<=count3;
						temp[count3]<=temp[count3];
					end
				end
				else begin
					count3<=3'd7;
					temp[count3]<=0;
				end
			end
		end
		else begin
			if(lsbfe_i) begin
				if(count2<=3'd7) begin
					if(miso_receive_sclk) begin
						count2<=count2+3'd1;
						temp[count2]<=miso_i;
					end
					else begin
						count2<=count2;
						temp[count2]<=temp[count2];
					end
				end
				else begin
					count2<=3'd0;
					temp[count2]<=0;
```

```
                        end
                    end
                    else begin
                        if(count3>=3'd0) begin
                            if(miso_receive_sclk) begin
                                count3<=count3-3'd1;
                                temp[count3]<=miso_i;
                            end
                            else begin
                                count3<=count3;
                                temp[count3]<=temp[count3];
                            end
                        end
                        else begin
                            count3<=3'd7;
                            temp[count3]<=0;
                        end
                    end
                end
            end
        end
    end

    //logic count,count1,shift_reg

        always@(posedge pclk or negedge presetn) begin
        if(!presetn)begin
            count<=3'd0;
            count1<=3'd7;
            mosi_o<=0;
        end
        else begin
            if(ss_i) begin
                count<=count;
                count1<=count1;
                mosi_o<=mosi_o;
            end
            else begin
                if(spimode) begin
                    if(lsbfe_i) begin
                        if(count<=3'd7) begin
                            if(mosi_send_sclk) begin
                                count<=count+3'd1;
                                mosi_o<=shift_reg[count];
                            end
                            else begin
                                count<=count;
                                mosi_o<=mosi_o;
                            end
                        end
                        else begin
                            count<=3'd0;
```

```
                        mosi_o<=0;
                    end
                end
                else begin
                    if(count1>=3'd0) begin
                        if(mosi_send_sclk) begin
                            count1<=count1-3'd1;
                            mosi_o<=shift_reg[count1];
                        end
                        else begin
                            count1<=count1;
                            mosi_o<=mosi_o;
                        end
                    end
                    else begin
                        count1<=3'd7;
                        mosi_o<=0;
                    end
                end
            end
            else begin
                if(lsbfe_i) begin
                    if(count<=3'd7) begin
                        if(mosi_send_sclk0) begin
                            count<=count+3'd1;
                            mosi_o<=shift_reg[count];
                        end
                        else begin
                            count<=count;
                            mosi_o<=mosi_o;
                        end
                    end
                    else begin
                        count<=3'd0;
                        mosi_o<=0;
                    end
                end
                else begin
                    if(count1>=3'd0) begin
                        if(mosi_send_sclk0) begin
                            count1<=count1-3'd1;
                            mosi_o<=shift_reg[count1];
                        end
                        else begin
                            count1<=count1;
                            mosi_o<=mosi_o;
                        end
                    end
                    else begin
                        count1<=3'd7;
                        mosi_o<=0;
                    end
```

```
                                        end
                                    end
                                end
                            end
                        end

endmodule
```

APB interface:

```
module
APB_slave(pclk,presetn,paddr_i,pwrite_i,psel_i,penable_i,pwdata_i,ss_i,miso_data,receive_data_i,t
ip_i,prdata,mstr_o,cpol_o,cpha_o,lsbfe_o,spiswai_o,sppr_o,spr_o,spi_interrupt_reqest,pready,pslve
rr,send_data_o,mosi_data_o,spi_mode_o);

        input pclk,presetn;
        input [2:0]paddr_i;
        input pwrite_i;
        input psel_i,penable_i;
        input [7:0]pwdata_i;
        input ss_i;
        input [7:0]miso_data;
        input receive_data_i,tip_i;

        output reg [7:0]prdata;
        output mstr_o,cpol_o,cpha_o,lsbfe_o; //control register 1
        output spiswai_o;//control regester 2
        output [2:0]sppr_o,spr_o; //baudrate register
        output reg spi_interrupt_reqest;
        output pready,pslverr;
        output reg send_data_o;
        output reg [7:0]mosi_data_o;
        output [1:0]spi_mode_o;

        reg [7:0] SPI_CR1,SPI_CR2,SPI_BR,SPI_DR;
        wire [7:0]SPI_SR;
        wire spie,spe,sptie,ssoe; //control register 1
        wire modf;
        wire modfen; //status register
        wire spif,sptef; //status register
        wire wr_enb,rd_enb;

        reg [2:0]state, next_state; //APB FSM
        reg [1:0]spi_mode_o,next_mode; //SPI FSM

        parameter idle=3'd3,
                setup=3'd4,
                enable=3'd5,
                spi_run=2'd0,
```

```
        spi_wait=2'd1,
        spi_stop=2'd2;

parameter cr2_mask=8'b00011011;
parameter br_mask=8'b01110111;

assign pready= (state==enable)? 1'b1:1'b0;
assign wr_enb= (state==enable && pwrite_i)? 1'b1:1'b0;
assign rd_enb= (state==enable && !pwrite_i)? 1'b1:1'b0;
assign pslverr= (state==enable)? (~tip_i): 1'b0;
assign SPI_SR= (!presetn)? 8'b00100000: {spif,1'b0,sptef,modf,4'd0};
assign modf= (~ss_i) & mstr_o & modfen & (~ssoe);
assign sptef= (SPI_DR == 8'd0)? 1'b1: 1'b0;
assign spif= (SPI_DR != 8'd0)? 1'b1:1'b0;

always @(*) begin
        if(!rd_enb)
                prdata=8'd0;
        else begin
                case(paddr_i)
                        3'd0:prdata=SPI_CR1;
                        3'd1:prdata=SPI_CR2;
                        3'd2:prdata=SPI_BR;
                        3'd3:prdata=SPI_SR;
                        3'd5:prdata=SPI_DR;
                        default:prdata=8'd0;
                endcase
        end
end


        assign lsbfe_o = SPI_CR1[0];
        assign ssoe = SPI_CR1[1];
        assign cpha_o = SPI_CR1[2];
        assign cpol_o = SPI_CR1[3];
        assign mstr_o = SPI_CR1[4];
        assign sptie = SPI_CR1[5];
        assign spe = SPI_CR1[6];
        assign spie = SPI_CR1[7];
        assign modfen = SPI_CR2[4];
        assign spiswai_o = SPI_CR2[1];
        assign sppr_o = SPI_BR[6:4];
        assign spr_o = SPI_BR[2:0];


always @(*) begin
        if(~spie & ~sptie)
                spi_interrupt_reqest=1'b0;
        else begin
                if(~sptie & spie)
                        spi_interrupt_reqest= spif | modf;
                else begin
```

```
                        if (sptie & ~spie)
                                spi_interrupt_reqest=sptef;
                        else
                                spi_interrupt_reqest=spif | modf | sptef;
                end
        end
end

always @(posedge pclk or negedge presetn) begin
        if(!presetn)
                state<=idle;
        else
                state<=next_state;
end

always @(*) begin
        next_state=idle;
        case(state)
                idle:begin
                        if(psel_i && !penable_i)
                                next_state=setup;
                        else
                                next_state=idle;
                end
                setup: begin
                        if(psel_i && !penable_i)
                                next_state=setup;
                        else if(psel_i && penable_i)
                                next_state=enable;
                        else
                                next_state=idle;
                end
                enable:begin
                        if(psel_i)
                                next_state=setup;
                        else
                                next_state=idle;
                end
        endcase
end

always @(posedge pclk or negedge presetn) begin
        if(!presetn)
                spi_mode_o<=spi_run;
        else
                spi_mode_o<=next_mode;
end

always @(*) begin
        next_mode=spi_run;
        case(spi_mode_o)
                spi_run:begin
```

```
                        if(!spe)
                                next_mode=spi_wait;
                        else
                                next_mode=spi_run;
                end
                spi_wait:begin
                        if(spiswai_o)
                                next_mode=spi_stop;
                        else if(spe)
                                next_mode=spi_run;
                        else
                                next_mode=spi_wait;
                end
                spi_stop:begin
                        if(!spiswai_o)
                                next_mode=spi_wait;
                        else if(spe)
                                next_mode=spi_run;
                        else
                                next_mode=spi_stop;
                end
        endcase
end

always @(posedge pclk or negedge presetn)begin
        if(!presetn)
                send_data_o<=0;
        else begin
                if(wr_enb)
                        send_data_o<=1'd0;
                else begin
                        if((SPI_DR == pwdata_i) && (SPI_DR != miso_data) &&
(spi_mode_o == spi_run || spi_mode_o == spi_wait))
                                send_data_o<=1'b1;
                        else
                                send_data_o<=1'd0;
                end
        end
end

always @(posedge pclk or negedge presetn)begin
        if(!presetn)
                mosi_data_o<=8'd0;
        else begin
                if((SPI_DR==pwdata_i) && (SPI_DR != miso_data) && (spi_mode_o ==
spi_run || spi_mode_o == spi_wait))
                        mosi_data_o<=SPI_DR;
                else
                        mosi_data_o<=mosi_data_o;
        end
end
```

```
	always @(posedge pclk or negedge presetn)begin
		if(!presetn) begin
			SPI_CR1<=8'h04;
			SPI_CR2<=8'd0;
			SPI_BR<=8'd0;
			SPI_DR<=8'd0;
		end
		else begin
			if(!wr_enb) begin
				SPI_CR1<=SPI_CR1;
				SPI_CR2<=SPI_CR2;
				SPI_BR<=SPI_BR;
				if((SPI_DR==pwdata_i) && (SPI_DR != miso_data) &&
(spi_mode_o == spi_run || spi_mode_o == spi_wait))
					SPI_DR<=8'd0;
				else begin
					if(receive_data_i && (spi_mode_o == spi_run || spi_mode_o
== spi_wait))
						SPI_DR<=miso_data;
					else
						SPI_DR<=SPI_DR;
				end
			end
			else begin
				SPI_CR1<= (paddr_i==3'd0)? pwdata_i : SPI_CR1;
				SPI_CR2<= (paddr_i==3'd1)? (pwdata_i & cr2_mask): SPI_CR2;
				SPI_BR<= (paddr_i==3'd2)? (pwdata_i & br_mask): SPI_BR;
				SPI_DR<= (paddr_i==3'd5)? pwdata_i:SPI_DR;
			end
		end
	end

endmodule
```

Top module:

```
module
top_module(pclk,presetn,paddr_i,psel_i,penable_i,pwrite_i,pwdata_i,miso_i,prdata_o,pready_o,pslv
err_o,mosi_o,ss_o,sclk_o,spi_interrupt_request);
	input pclk,presetn;
	input [2:0]paddr_i;
	input psel_i,penable_i,pwrite_i;
	input [7:0]pwdata_i;
	input miso_i;

	output [7:0] prdata_o;
	output pready_o,pslverr_o,mosi_o;
	output ss_o,sclk_o,spi_interrupt_request;
```

```
	wire [1:0]spi_mode_t;
	wire
spiswai_t,cpol_t,cpha_t,mosi_send_sclk_t,mosi_send_sclk0_t,miso_recive_sclk_t,miso_recive_sclk
0_t;
	wire [11:0]BaudrateDivisor_t;
	wire mstr_t;
	wire [2:0]sppr_t,spr_t;
	wire send_data_t,receive_data_t,tip_t,lsbfe_t;
	wire [7:0]data_mosi_t;
	wire [7:0]data_miso_t;


	baudrate_gen baud_rate(.pclk(pclk),
		.presetn(presetn),
		.spi_mode_i(spi_mode_t),
		.spiswai_i(spiswai_t),
		.sppr_i(sppr_t),
		.spr_i(spr_t),
		.cpol_i(cpol_t),
		.cpha_i(cpha_t),
		.ss_i(ss_o),
		.sclk(sclk_o),
		.mosi_send_sclk_o(mosi_send_sclk_t),
		.mosi_send_sclk0_o(mosi_send_sclk0_t),
		.miso_recive_sclk_o(miso_recive_sclk_t),
		.miso_recive_sclk0_o(miso_recive_sclk0_t),
		.BaudrateDivisor_o(BaudrateDivisor_t));


	spi_slave_select slave_select(.pclk(pclk),
		.presetn(presetn),
		.mstr_i(mstr_t),
		.spiswai_i(spiswai_t),
		.spimode_i(spi_mode_t),
		.send_data_i(send_data_t),
		.baudRateDivisor_i(BaudrateDivisor_t),
		.receive_data_o(receive_data_t),
		.ss_o(ss_o),
		.tip_o(tip_t));


	shift_reg shift_register(.pclk(pclk),
		.presetn(presetn),
		.ss_i(ss_o),
		.send_data_i(send_data_t),
		.lsbfe_i(lsbfe_t),
		.cpha_i(cpha_t),
		.cpol_i(cpol_t),
		.miso_receive_sclk0(miso_recive_sclk0_t),
		.miso_receive_sclk(miso_recive_sclk_t),
		.mosi_send_sclk0(mosi_send_sclk0_t),
		.mosi_send_sclk(mosi_send_sclk_t),
```

```
                .data_mosi_i(data_mosi_t),
                .miso_i(miso_i),
                .receive_data_i(receive_data_t),
                .mosi_o(mosi_o),
                .data_miso_o(data_miso_t));

        APB_slave slave_interface(.pclk(pclk),
                .presetn(presetn),
                .paddr_i(paddr_i),
                .pwrite_i(pwrite_i),
                .psel_i(psel_i),
                .penable_i(penable_i),
                .pwdata_i(pwdata_i),
                .ss_i(ss_o),
                .miso_data(data_miso_t),
                .receive_data_i(receive_data_t),
                .tip_i(tip_t),
                .prdata(prdata_o),
                .mstr_o(mstr_t),
                .cpol_o(cpol_t),
                .cpha_o(cpha_t),
                .lsbfe_o(lsbfe_t),
                .spiswai_o(spiswai_t),
                .sppr_o(sppr_t),
                .spr_o(spr_t),
                .spi_interrupt_reqest(spi_interrupt_request),
                .pready(pready_o),
                .pslverr(pslverr_o),
                .send_data_o(send_data_t),
                .mosi_data_o(data_mosi_t),
                .spi_mode_o(spi_mode_t));


endmodule
```

Top module Test Bench:

```
module top_module_tb();
        reg pclk,presetn;
        reg [2:0]paddr_i;
        reg psel_i,penable_i,pwrite_i;
        reg [7:0]pwdata_i;
        reg miso_i;

        wire [7:0] prdata_o;
        wire pready_o,pslverr_o,mosi_o;
        wire ss_o,sclk_o,spi_interrupt_request;
```

```
	top_module
dut(pclk,presetn,paddr_i,psel_i,penable_i,pwrite_i,pwdata_i,miso_i,prdata_o,pready_o,pslverr_o,m
osi_o,ss_o,sclk_o,spi_interrupt_request);


	always begin
		#20 pclk=1'b0;	//25MHz freq pclk timeperiod=40ns
		#20 pclk=~pclk;
	end

	task reset();
		begin
			@(negedge pclk);
			presetn=1'b0;
			@(negedge pclk);
			presetn=1'b1;
		end
	endtask

	task spi_config_reg(input [7:0]data, input [2:0]addr);
		begin
			@(negedge pclk);
			paddr_i=addr;
			pwdata_i=data;
			psel_i=1'b1;
			penable_i=1'b0;
			pwrite_i=1'b1;
			@(negedge pclk);
			penable_i=1'b1;
			wait(!pready_o)
			@(negedge pclk);
			psel_i=1'b0;
			penable_i=1'b0;
			@(negedge pclk);
		end
	endtask

	task spi_config_sr(input [2:0]addr);
		begin
			@(negedge pclk);
			paddr_i=addr;
			psel_i=1'b1;
			penable_i=1'b0;
			pwrite_i=1'b0;
			@(negedge pclk);
			penable_i=1'b1;
			wait(!pready_o)
			@(negedge pclk);
			psel_i=1'b0;
			penable_i=1'b0;
			@(negedge pclk);
		end
```

```
	endtask

	task initialize();
		begin
			{presetn,paddr_i,psel_i,penable_i,pwrite_i,pwdata_i,miso_i}=0;
		end
	endtask

	initial begin
		initialize();
		reset();
		miso_i=1'b1;
		spi_config_reg(8'b0101_0111,3'd0);
		spi_config_reg(8'b01010101,3'd1);
		spi_config_reg(8'b0000_0000,3'd2);
		spi_config_reg(8'b1101_1011,3'd5);
		spi_config_sr(3'd3);
		spi_config_sr(3'd0);
		spi_config_sr(3'd1);
		spi_config_sr(3'd2);
		spi_config_sr(3'd5);
	end

	initial begin
		#5000;
		$finish;
	end

endmodule
```